Cost-Aware Agent RL: Implementation Blueprint

Oct 7, 2026 · @vsv

With unlimited compute, run online GRPO on a Gemma 4 31B LoRA under a verifiable, success-gated cost reward, add step-level DPO and a Laya router, and publish a Pareto frontier of resolve rate against repository interactions by November 12.

Status. Nothing in this document has been run. Where it says what a component improves, that is the effect we expect and the experiment arm that checks it. None of it is a measured result.

What unlimited compute changes

The bottleneck moves from GPU hours to wall-clock time (5 weeks to the November 12 deadline) and to sandbox throughput, so the plan now uses online RL instead of offline-only training.

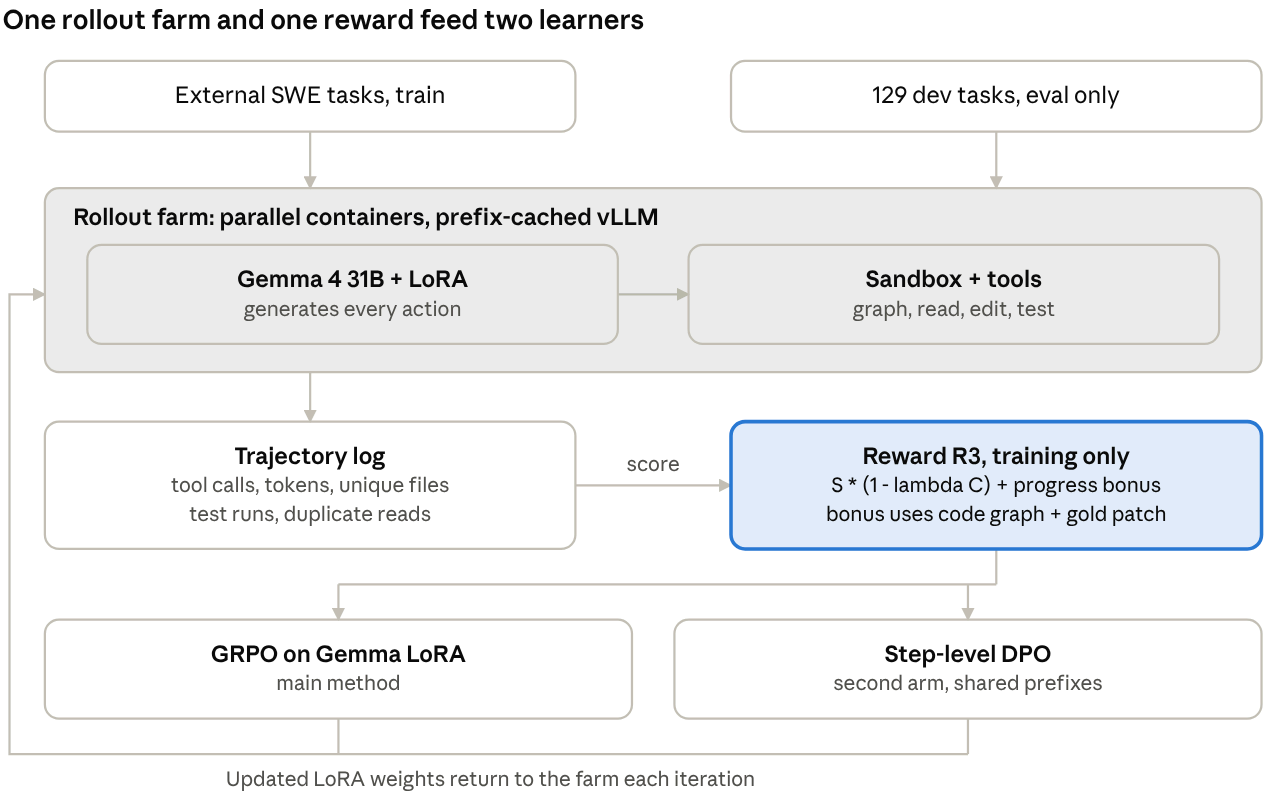
Constraint	Limited-compute plan	Unlimited-compute plan
Main RL method	Offline step-DPO	Online GRPO on the Gemma LoRA, plus step-DPO as a second arm
Rollouts per task	4	16 to 64 per task, with prefix-cached branching
Training tasks	300 to 800 external tasks	5,000+ from SWE-smith, SWE-Gym and SWE-rebench V2
Reward variants	R1, R2, R3 at one lambda	R1, R2, R3 across a 6-point lambda sweep, 3 seeds each
Step labels	Cheap local value, few true rollouts	True Monte Carlo Q for every branched step, m = 8
Router	SFT plus small GRPO	Full GRPO, joint and alternating training with the Gemma policy
Evaluation	One run on 129 dev tasks	5 samples per task, leave-one-repo-out, full Pareto with confidence intervals

Three things stay fixed by the rules: every agent and subagent must use `gemma-4-31b-it-qat-w4a16-ct`, LoRA adapters are optional, and the paper is capped at 3,000 words. Hidden test tasks come from private repositories, so train on external data and treat the 129 public dev tasks (FastAPI, Requests, Rich, HTTPX) as evaluation only.

What each upgrade buys.

- **Online GRPO.** The model learns from its own current mistakes instead of only from pairs generated by an older version of itself.
- **16 to 64 rollouts per task.** More attempts per issue gives a more reliable comparison between them, and a better chance of seeing at least one solve on hard tasks.
- **5,000+ training tasks.** Many tasks stop the agent from memorizing a few repositories. The leave-one-repo-out test checks this.
- **Reward variants, lambda sweep, seeds.** The variants answer which reward design matters, the lambda sweep draws the Pareto frontier, and the seeds show how much is noise.
- **True Monte Carlo value (m = 8).** Pair labels come from real outcomes instead of a cheap proxy, so fewer pairs are mislabeled.
- **Full router GRPO.** The router learns from real outcomes instead of only copying labels.

Architecture



training loop · 1 reward, 2 learners

Every rollout is scored by one reward. The two learners update the Gemma LoRA, and the updated weights return to the farm for the next iteration. The Laya router is left out for now.

What each box does and why it is there.

Box	What it does	Why it is there
External SWE tasks, train	Supplies issues, repositories, tests and reference patches	The hidden test set comes from private repositories, so training must use other data
129 dev tasks, eval only	Held-out tasks from FastAPI, Requests, Rich and HTTPX	Measures generalization; never used for training
Rollout farm	Runs many attempts in parallel in isolated containers, with prefix-cached vLLM	RL needs thousands of attempts; prefix caching reuses computed context when branching from the same state

Box	What it does	Why it is there
Gemma 4 31B + LoRA	The policy: it writes every action	The competition requires this model; a LoRA adapter trains a small set of extra weights instead of all 31B
Sandbox + tools	Executes each tool call, runs tests, returns observations	Gives the agent real feedback and gives us the outcome S
Trajectory log	Records every step with its token and tool costs	Source for both the reward and the preference pairs
Reward R3, training only	Scores each attempt from cost, success, the code graph and the gold patch	The learning signal; the gold patch stays hidden from the agent
GRPO on Gemma LoRA	Compares attempts per task and shifts the policy toward the better ones	Main method
Step-level DPO	Learns from better-versus-worse action pairs at a single state	Second arm; tests whether step-level labels beat attempt-level rewards
Feedback loop	Sends updated weights back to the farm	The next rollouts come from the improved policy

Formulation

Treat issue resolution as a constrained MDP: maximize solve probability while holding normalized interaction cost under a budget, then sweep the budget to trace the Pareto frontier.

MDP. For task q (issue plus repository snapshot):

- State: s_t = (q, h_t), where h_t is the history of actions and observations.
- Action a_t: one of search, graph_query, read(file, line range), edit, test(target), submit. Gemma generates the tokens.
- Trajectory: tau = (s_0, a_0, o_0, ..., s_T).
- Outcome S(tau) in {0, 1}: whether the hidden validation tests pass.

Normalized cost. A hard task legitimately needs more interactions, so each cost type is scaled by the median over the base policy's successful runs on the same task:

```
C(\tau)=\sum_{k} w_k\,\frac{\min\!\big(n_k(\tau)/\bar n_k(q),\,c_{\max}\big)}
{c_{\max}}\in[0,1],\qquad \sum_k w_k=1
```

The cost types k are tool calls, tokens (generated plus observation), unique files, test runs and duplicate reads. Counting observation tokens is what teaches the agent to read line ranges instead of whole files. When the base policy never solves q, fall back to the global median of n_k.

Constrained objective and its Lagrangian.

```
\max_{\pi}\ \mathbb{E}_{\pi}[S(\tau)]\quad\text{s.t.}\quad \mathbb{E}_{\pi}
[C(\tau)]\le B
\qquad\Longleftrightarrow\qquad
\max_{\pi}\min_{\lambda\ge 0}\ \mathbb{E}_{\pi}\big[S-\lambda\,(C-B)\big]
```

Why not the plain weighted sum. The reward S - lambda C pays a short failure more than a long failure, so on hard tasks the agent learns to give up early. The success-gated form below removes that incentive.

Term by term: what the formulation says and why

The decision-process pieces.

Symbol	Meaning	Purpose
q	One task: the issue text plus a frozen copy of the repository	Everything the agent is asked to fix
s_t	State at step t: q plus everything the agent has done and seen so far	The agent chooses each action from this, so memory of the past is part of the state
a_t	Action at step t: a tool call such as search or read, written as Gemma tokens	What the policy chooses and RL changes
o_t	Observation the tool returned (file text, test output)	New information. It costs tokens and is not chosen by the model

Symbol	Meaning	Purpose
tau	The whole trajectory of actions and observations for one attempt	The unit that receives a final score and a cost
S(tau)	1 if the hidden tests pass after the final patch, else 0	The only real measure of success, verified by running code and not judged by a model
pi	The policy: the agent's rule for picking a_t from s_t	What we are optimizing

Why sequential decisions. Each file the agent opens changes what it knows, so early actions decide what later actions are possible. A one-shot patch generator cannot express "look less". A sequential model can, because looking is an action with a price.

The cost C(tau), term by term.

Term	Meaning	Purpose
k	Index over cost types: tool calls, tokens, unique files, test runs, duplicate reads	One number covers several kinds of waste
n_k(tau)	Raw count of cost type k in this trajectory. A duplicate read means re-reading a file span already seen with no edit in between	What the agent actually spent
n-bar_k(q)	Median count of type k over the base model's successful runs on this same task. If the base never solves it, use the global median	Defines normal effort for this task, so hard tasks are not punished for needing more steps
n_k divided by n-bar_k	Ratio to normal: 1 is typical, 2 is double	Makes tokens, calls and tests comparable despite different units
min(ratio, c_max)	The ratio capped at c_max, set near the 90th to 95th percentile of baseline ratios	One runaway trajectory cannot dominate the reward
Divide by c_max	Rescales each term to 0 to 1	Keeps the total between 0 and 1

Term	Meaning	Purpose
w_k	Weight on cost type k, fitted by regressing measured GPU-seconds on the counts from baseline rollouts, then normalized	Charges each resource in proportion to what it really costs, not a guess
Sum of w_k equals 1	Weights add to one	Guarantees C stays in 0 to 1, so lambda has a fixed meaning

What it improves: a trajectory is judged against what that task normally needs, not against an arbitrary global budget, and the cap stops one outlier from distorting training. The weights and the cap are design choices to calibrate and ablate, not derived constants. Duplicate reads get a small weight (about 0.05), because some re-reading is harmless.

The constrained objective, term by term.

Term	Meaning	Purpose
max over pi	Search over agent behaviors	We want the best policy
E_pi[S]	Average success when the agent follows pi	Resolve rate: the thing we must not lose
E_pi[C]	Average normalized cost	The thing we want to shrink
B	Cost budget between 0 and 1	How much effort we allow on average
Constraint E[C] at most B	Cost may not exceed the budget	Turns "solve cheaply" into a precise question: best success at or under budget B
lambda	Lagrange multiplier, at least 0: the price of one unit of cost above budget	Converts the constraint into a reward the agent can learn from
S minus lambda times (C minus B)	The Lagrangian	Reward for success, charged for cost above budget
max over pi, min over lambda	The policy tries to raise it, lambda rises until the budget is met	At the optimum, lambda is the exchange rate between success and cost

What it improves: sweeping B or lambda draws the whole Pareto frontier of success versus cost instead of one arbitrary trade-off. In practice we fix lambda for each run and sweep it, because learning lambda online adds instability.

Why not the plain weighted sum, with numbers. Take lambda 0.5. A failed run that gave up after three calls has C = 0.1 and scores -0.05. A failed run that tried hard has C = 0.9 and scores -0.45. The agent learns that failing fast beats trying, which is the wrong lesson on hard tasks. Gating the cost behind success (R2) removes it: every failure scores 0 whatever the effort.

Reward formulations

R3 is the proposed method: a success-gated cost plus a potential-based localization bonus computed from the code graph and the gold patch, with no learned reward model.

Name	Reward	What it tests
R1	S	Outcome-only baseline
R2	S * (1 - lambda * C), lambda in [0, 1)	Cost-aware, any success beats any failure
R2-ungated (ablation)	S - lambda * C	Whether early-quit collapse appears
R3	R2 + eta * sum_t [gamma * Phi(s_t+1) - Phi(s_t)]	Dense, verifiable progress credit

Verifiable process potential. Let S* be the set of symbols and files edited by the reference patch, and d_G the shortest-path distance in the provided `graphs/` code graph. The potential rewards the closest node the agent has seen so far:

```
\Phi(s)=\max_{v\in \mathrm{seen}(s)}\
\gamma_g^{\,d_G(v,\,\mathcal{S}^*)},\qquad \gamma_g\in(0,1)
```

Because the shaping term is potential-based (Ng, Harada and Russell, 1999), it leaves the optimal policy unchanged and only changes how credit is assigned across steps (see the caveat on dense credit below). The gold patch is used for training rewards only and is never shown to the agent. This replaces the learned process reward model in PRLCoder, which reached about 75% accuracy and was blamed there for PPO trailing DPO.

Step-level value for preference pairs. At a state s_t, sample K candidate actions from the base policy, roll each out m times, and estimate:

```
\hat Q(s_t,a)=\frac1m\sum_{j=1}^{m} R(\tau_j),\qquad a^{+}=\arg\max_a \hat Q,\quad a^{-}=\arg\min_a \hat Q
```

Keep a pair only when Q(a+) - Q(a-) exceeds a margin delta and a+ led to at least one solve. With unlimited compute, use K = 8 and m = 8 at every branch point.

Term by term: what each reward does

The four rewards differ by one idea each, so comparing them isolates that idea.

R1 = S. One if the hidden tests pass, zero otherwise. It is the baseline. Its weakness is that an 11-call solve and a 38-call solve score identically, so training has no pressure toward efficiency.

R2 = S times (1 minus lambda times C).

Term	Meaning	Purpose
S	1 if solved, 0 if not	Gate: cost only matters when the task is solved
C	Normalized cost between 0 and 1	How much extra effort the solve took
lambda	Share of the reward given up at maximum cost, at least 0 and below 1	The efficiency pressure. Zero gives back R1
1 minus lambda times C	Discount factor	Cheaper solves keep more of their reward
The product with S	Any failure scores exactly 0	The agent cannot gain by quitting early

Example with lambda 0.5: a solve at cost 0.2 scores 0.9, a solve at cost 0.6 scores 0.7, and every failure scores 0. Because 1 minus lambda is positive, any solve beats any failure. What it improves: among successful attempts, the cheaper ones get a larger advantage in GRPO, so the policy drifts toward efficient solves without learning to give up.

R2-ungated = S minus lambda times C. Same terms, but cost is also charged on failures. It exists to show the problem: a failure that gives up early scores better than one that tried hard, so we expect lower solve rates on hard tasks.

R3 = R2 plus eta times the progress bonus.

Term	Meaning	Purpose
eta	Weight on the progress bonus	Keeps the bonus small. If eta is below 1 minus lambda, even a failure that localized perfectly scores below the worst solve
t	Step index within the trajectory	Position in the sequence
Phi of s_t	Progress score at state s_t: how close the nearest code the agent has seen is to the code the real fix edits	Tells us a step helped before any test runs
gamma	Discount inside the shaping term; 1.0 in the code skeleton	Sets how much later progress counts against earlier progress
gamma times Phi(s_t+1) minus Phi(s_t)	Gain in progress caused by step t	Positive when the step revealed code closer to the fix, zero when it revealed nothing new
Sum over t	Adds the gains over the trajectory	The total bonus

The progress score Phi, term by term.

Term	Meaning	Purpose
seen(s)	Symbols and files the agent has opened or queried so far	What the agent actually knows
v	One node in the code graph: a function, class or file	A candidate piece of code the agent looked at

Term	Meaning	Purpose
S*	Symbols and files the reference patch edits	The target. Used only for training rewards, never shown to the agent
d_G(v, S*)	Number of graph hops from v to the nearest node in S*	0 means exactly the right code, 1 a direct caller or callee, higher means farther
gamma_g	Decay per hop, between 0 and 1, for example 0.5	Credit 1.0 at the fix, 0.5 one hop away, 0.25 two hops, 0.125 three hops
max over v in seen(s)	The best node seen so far	Phi never decreases, so re-reading adds nothing and unrelated reads are not rewarded here

Intuition. R2 pays once, at the end. Nothing in it says that opening token.py was a good move and opening ten unrelated files was not. Phi supplies that missing signal from structure we already have: the code graph and the reference patch. It replaces the learned process reward model in PRLCoder, which was only about 75% accurate, with a check that is exact on training tasks.

Worked example. The fix edits validate_token in token.py. The agent searches and opens auth.py, two hops away: Phi 0.25. It then opens token.py: Phi 1.0. Later reads add nothing. The bonus is eta times 1.0.

Caveat on dense credit. With gamma 1 and one reward for a whole attempt, the summed bonus collapses to eta times (final Phi minus starting Phi). That rewards how close the agent got, not how fast, so it works as a localization-quality bonus. Step-by-step credit appears only if advantages are computed per step, for example from the return-to-go at each step. Plan R3 both ways and compare. An earlier version of this document overstated this.

Step-level value for preference pairs, term by term.

Term	Meaning	Purpose
s_t	A state chosen as a branch point, preferably just before the first correct localization	Where the choice between actions mattered

Term	Meaning	Purpose
K	Number of candidate next actions sampled there (8 planned)	The options being compared
m	Rollouts run from each candidate (8 planned)	Averages out luck
R(tau_j)	Reward of rollout j	Outcome of continuing after that action
Q-hat(s_t, a)	Average reward over the m rollouts	Estimate of how good action a is at that state
a+ and a-	Actions with the highest and lowest Q-hat	Chosen and rejected sides of the pair
delta	Minimum gap between Q-hat(a+) and Q-hat(a-)	Drops pairs that differ only by noise
At least one solve for a+	Extra requirement on the chosen side	Prevents teaching the agent to prefer the less bad of two failures

What it improves: step-level labels with no learned reward model, which is the PRLCoder idea. Cost: K times m rollouts per branch state, 64 with the planned settings.

Training stack

Three learners share one rollout farm: online GRPO on the Gemma LoRA (main), step-level DPO on the same LoRA (second arm), and GRPO on the Laya router (System 1).

1. Warm start. Run rejection-sampling fine-tuning (RFT) first: sample 16 to 64 trajectories per training task, keep the successful ones with the lowest cost C, and fine-tune the LoRA on them. This is also your strongest non-RL baseline, so reviewers will ask for it.

2. Online GRPO on the Gemma LoRA (main method). For each task, sample G trajectories from the current policy, score them with R1, R2 or R3, and compute group-relative advantages:

```
A_i=\frac{R_i-\bar R}{\sigma_R+10^{-6}},\qquad
\mathcal{L}(\theta)=-\,\mathbb{E}\Big[\tfrac{1}{\sum_i
|a_i|}\sum_i\sum_{t}\min\!\big(\rho_{i,t}A_i,\ \mathrm{clip}(\rho_{i,t},1\!-
\!\epsilon,1\!+\!\epsilon)A_i\big)\Big]+\beta\,\mathrm{KL}
(\pi_\theta\,\|\,\pi_{\mathrm{ref}})
```

Here rho is the per-token importance ratio, summed only over tokens Gemma generated (tool outputs are masked out). GRPO needs no value model, which removes the critic that PPO would need for a 31B policy. Use G = 16, 5,000+ tasks, prefix caching in vLLM, and a framework that supports multi-turn tool use with LoRA (verl, SkyRL or TRL; verify before committing).

3. Step-level DPO (second arm, the PRLCoder analogue). Build shared-prefix pairs from the branching procedure in the reward section, then train with a masked DPO loss plus an NLL anchor on the chosen action (as in RPO) to fight likelihood displacement:

```
\mathcal{L}=-\log\sigma\!\Big(\beta\Big[\log\tfrac{\pi_\theta(a^{+}|s_t)}
{\pi_{\mathrm{ref}}(a^{+}|s_t)}-\log\tfrac{\pi_\theta(a^{-}|s_t)}
{\pi_{\mathrm{ref}}(a^{-}|s_t)}\Big]\Big)+\alpha\,\mathrm{NLL}(a^{+}|s_t)
```

Use two pair types in roughly equal numbers: efficiency pairs (both solve, chosen has lower C) and competence pairs (chosen solves, rejected does not). Efficiency pairs alone risk trading away solve rate. Trajectory-level DPO is not strictly valid for multi-turn agents because the two trajectories visit different states; shared-prefix step pairs avoid that, which is worth one sentence in the paper.

4. Alternating training. After each policy update, regenerate rollouts and retrain the router on them, so the router never lags the policy it directs.

5. Adapter transfer check. The competition model is the quantized `gemma-4-31b-it-qat-w4a16-ct`. Train on whatever precision your framework supports, then verify on day 1 that the adapter still helps on the quantized checkpoint before spending more compute.

How GRPO and DPO learn, term by term

Both methods push the Gemma LoRA toward cheaper successful trajectories. GRPO learns from fresh attempts scored by the reward; DPO learns from stored pairs of a better and a worse action at the same state.

GRPO: compare attempts, reinforce the better ones

For one issue the agent makes G independent attempts and each is scored. Tokens from an attempt that beat its siblings become more likely, tokens from one that lost become less likely, in small safe steps.

Term	What it is	Purpose
G	Number of attempts at the same task (16 planned)	Forms a comparison group for that task
R_i	Reward of attempt i, from R1, R2 or R3	The score being optimized
R-bar	Mean reward of the group	Baseline that replaces the value model PPO would need
sigma_R	Standard deviation of the group's rewards	Puts advantages on one scale across easy and hard tasks
10^-6	Tiny constant in the denominator	Avoids dividing by zero when all attempts tie; the advantage is then zero and the task teaches nothing
A_i	(R_i - R-bar) / sigma_R, the advantage	Positive means better than its siblings, so raise the probability of its actions; negative means lower it
rho_i,t	New-policy probability of token t divided by the probability under the policy that sampled it	Corrects for the policy changing between sampling and updating
clip(rho, 1 - eps, 1 + eps)	rho forced into a narrow band around 1; eps is the clip range, commonly 0.2	Limits how far one update can move any token
min(rho * A, clip(rho) * A)	The smaller of the unclipped and clipped terms	Pessimistic trust region: no extra credit for moving beyond the band

Term	What it is	Purpose
Sum over attempts and tokens, divided by total tokens	Average over every token Gemma wrote; tool outputs are masked out	Trains only on choices the model made, not on what the environment returned
beta * KL(pi_theta, pi_ref)	Penalty for drifting from the RFT starting policy	Stops collapse into odd behavior or reward exploitation; beta sets how tight the leash is
E and the leading minus	Average over tasks and attempts; the loss is minimized while the objective is maximized	Standard sign and averaging convention

Worked example. Four attempts at one task score 0.9, 0.7, 0.0 and 0.0 (two solves, one cheaper than the other, and two failures; with lambda 0.5 these are solves at cost 0.2 and 0.6). The mean is 0.4 and the standard deviation 0.41, so the advantages are about +1.23, +0.74, -0.99 and -0.99. The cheap solve is pushed up hardest, the costly solve a little, and both failures are pushed down.

DPO: learn which action is better at one state

At a state where the agent had several options, the branched rollouts say which action led to better outcomes. DPO raises the probability of the better action and lowers the worse one, relative to the starting model, with no reward model and no new sampling during training.

Term	What it is	Purpose
s	Shared state: the issue plus history up to the branch point	Both actions are judged from identical context
a+ and a-	Chosen and rejected next action (its generated tokens), taken from the branches with higher and lower estimated value	The preference label
pi_theta	The model being trained	The thing that changes

Term	What it is	Purpose
pi_ref	Frozen copy of the starting model after RFT	Fixed yardstick for how far training has moved
log of pi_theta over pi_ref for a+	How much more likely training made the chosen action	Implicit reward for a+
Same for a-	How much more likely training made the rejected action	Implicit reward for a-
The difference inside the brackets	Margin between the two implicit rewards	Positive when the model moved toward a+ and away from a-; the quantity to grow
beta	Scale on the margin; larger values tie the model closer to pi_ref	Controls how far the model may drift from the starting point
Sigma and the negative log	Logistic probability that the chosen action wins, then its negative log	Loss shrinks as the margin grows; pairs already ranked correctly add little, wrongly ranked pairs add the most
alpha * NLL of a+	Ordinary next-token loss on the chosen action	Anchor against likelihood displacement, where DPO lowers both actions and the chosen one ends up less likely

Example pair (illustrative). At a state with nothing opened yet, a+ is searching for the failing function and reading only that span, while a- is opening three whole files first. Both branches were rolled out; the first led to a cheaper solve, so it is chosen.

How they differ here

	GRPO	Step-level DPO
Data	Fresh rollouts from the current model	Stored pairs from branched rollouts
Signal	One reward per whole attempt, shared across its tokens	Which of two actions is better at a single state
Credit assignment	Coarse, whole trajectory	Fine, single step

	GRPO	Step-level DPO
Compute during training	Needs live rollouts every update	Cheap once the pairs exist
Main risk	Reward hacking, training instability	Likelihood displacement, stale pairs as the model moves
Role here	Main method	Second arm, tests the PRLCoder finding

Why the pipeline has each stage

Stage	What it does	Without it	Expected improvement, and the check
RFT warm start	Fine-tunes on the cheapest successful trajectories	RL starts from a model that rarely solves hard tasks, so most rewards are zero and nothing is learned	A usable starting solve rate with cost already lowered. Arm A2 is also the simple baseline
GRPO	Reinforces the better attempts for each task	No improvement beyond the warm start	Lower cost at equal solve rate. Arms A3 to A5
Step-level DPO	Learns better-versus-worse actions at one state	No step-level signal	Corrects specific wasteful habits more cheaply. Arm A6
KL penalty (beta in GRPO)	Pulls the policy back toward the RFT model	The model can drift into exploiting the reward	Stable training. Watch the KL value and treat a spike as a warning
NLL anchor (alpha in DPO)	Keeps the chosen action likely	DPO can lower both actions and leave the chosen one less likely	Fewer collapsed adapters. Ablation "no NLL anchor"

Stage	What it does	Without it	Expected improvement, and the check
Efficiency plus competence pairs	Mixes cheaper-versus-costlier solves with solve-versus-fail	Efficiency pairs alone can trade away solve rate	Solve rate holds while cost falls. Ablation "efficiency-only pairs"
Token masking	Trains only on tokens Gemma wrote	The model learns to imitate tool output	Cleaner gradients
Alternating router and policy updates	Retrains the router on the new policy's rollouts	The router decides from behavior the policy no longer has	A router that stays matched. Compare with a router trained once
Adapter transfer check	Tests the adapter on the quantized competition model	An adapter that helps in training may not help in submission	Caught on day 1 instead of at the end

Laya / Jev router (System 1)

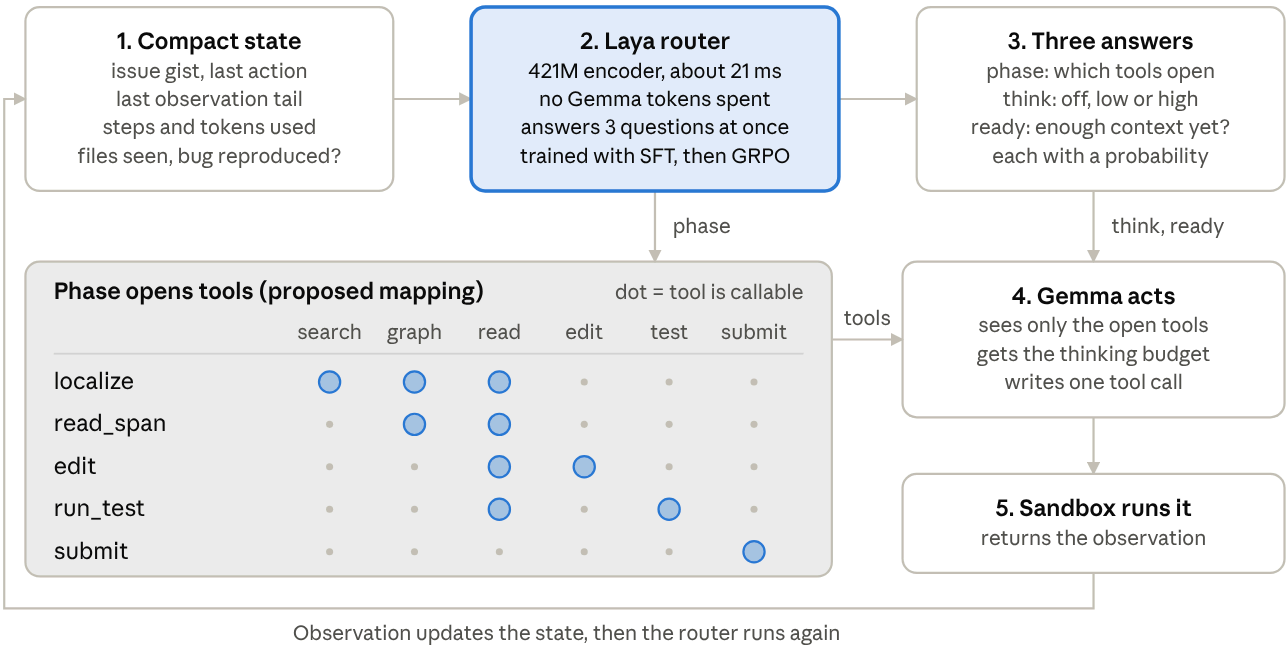
A 421M-parameter encoder decides the next phase and the thinking budget in about 21 ms with zero LLM tokens, so every step it routes saves reasoning tokens directly.

What the router is for. A 31B model spends similar effort on every step unless something tells it otherwise. The router adds that control with a small encoder instead of Gemma tokens:

- **Reasoning budget.** `think` sets off, low or high for the next step, so reads and test runs do not trigger long chains of thought.
- **Order of work.** `phase` opens only the tools that fit the current stage, so the agent cannot run tests before localizing or drift back into search while editing.
- **Stopping and starting.** `ready_to_edit` holds back edits until enough context is gathered and nudges the agent to commit once it stalls in reading.
- **Free decisions.** The call costs no Gemma tokens and about 21 ms, so routing every step is affordable.

These are hypotheses. Comparing arm A7 against the best arm without the router tests whether they hold.

The router picks the phase and thinking budget before Gemma writes a token

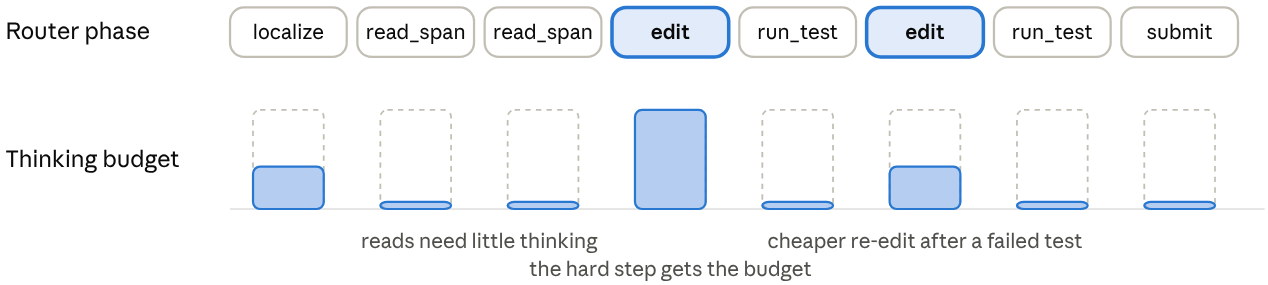


router step · state in, tools and budget out

Left to right across the top, a compact state goes through the router and comes out as three answers. The phase answer opens a subset of tools, think and ready go to Gemma, and the tool result updates the state for the next step. The tool mapping is a proposal, not a fixed design.

Illustrative rollout: thinking is spent mostly on the edit steps

Design intent for one eight-step rollout, not a measurement



Bar height = thinking budget: stub is off, medium is low, tall is high. Dashed outline = always-on thinking.

illustrative rollout · thinking by step

Read the dashed outline as always-on thinking and the filled bar as the router's choice; the gap is the saving the experiment has to measure.

Jev is TypeSafe AI's hosted decision model (released September 2026); Laya is the open Apache-2.0 replica with a Jev-compatible API. The `laya-conductor` fine-tune is the closest precedent: it routes coding-agent turns at 87% accuracy after about 30 minutes of training on a T4, using a GRPO plus soft cross-entropy recipe. Laya has no input-length headroom to waste: the conductor card caps the request field near 400 characters and the context at 1,024 tokens, so state must be compact.

Questions asked at every step

Question	Type	Options	Effect on Gemma
phase	choice	localize, read_span, edit, run_test, submit	Restricts the tool set for that step
think	choice	off, low, high	Sets the reasoning-token budget
ready_to_edit	noul	P(enough context gathered)	Blocks premature edits, or forces one after a stall

Router state (under 400 characters). Issue gist, last action, tail of the last observation, steps used, tokens used, number of unique files seen, whether the failure has been reproduced.

Why each state field is there.

Field	Purpose
Issue gist	Tells the router what kind of task this is
Last action	Prevents repeating the same step
Tail of the last observation	Shows what just happened, such as a test failure
Steps and tokens used	Lets the router tighten the thinking budget as the budget runs down
Unique files seen	A rough measure of how much exploring is done
Whether the failure has been reproduced	Editing before reproducing the bug is likely premature

Training.

1. Distill labels from the chosen trajectories: each step's realized phase and thinking level.
2. Fine-tune Laya with the conductor recipe (4 epochs, encoder lr 2.5e-5, head lr 1e-4).
3. Run GRPO with the router sampling its choices inside live rollouts, reward R3, and a KL penalty to the SFT router.

```
\mathcal{L}_\phi=-\sum_i\sum_t A_i\,\log\pi_\phi(m_t^{(i)}\mid
z_t^{(i)})+\beta\,\mathrm{KL}(\pi_\phi\,\|\,\pi_{\mathrm{SFT}})
```

The router loss, term by term.

Term	Meaning	Purpose
i	One rollout in the group for a task	Compare rollouts of the same task
t	One step in that rollout	The router decides at every step
z_t	Compact state shown to the router	What the router bases its decision on
m_t	The router's answers at step t: phase, think level, ready	The decision being learned
pi_phi(m_t given z_t)	Probability the router gave those answers	Raising it makes the same choice more likely next time
A_i	Advantage of rollout i, computed from its reward the same way as in GRPO	Positive if the rollout solved the task more cheaply than its siblings
Minus sign and the log	The gradient raises the log-probability when A_i is positive and lowers it when negative	Reinforces routing choices that appeared in cheap solves
beta times KL(pi_phi, pi_SFT)	Penalty for moving away from the router trained on labels	Keeps the router from drifting into odd behavior

What it improves: the label-trained router only imitates past behavior. GRPO adjusts it using real outcomes, so it can learn, for example, that low thinking on a read step saved tokens without hurting the solve. Check: arm A7 against the best arm without the router, and the Gemma-LoRA router in arm A8.

Rule risk. In the prediction track every agent and subagent must use the Gemma model, and only harness tools or custom agent_tool subagents are allowed, so Laya itself is probably not submittable there. For a submission, distill the router into a small Gemma LoRA subagent. In the paper, the comparison of Laya (zero tokens) against a Gemma-LoRA router is a clean ablation. Check the full rules page before committing, since this summary comes from a secondary source.

Experiments and metrics

The headline claim to test is non-inferiority: the same resolve rate within a stated margin, at a fraction of the interactions. With 129 dev tasks, a 3-point solve-rate gain will not be significant, but a cost reduction at equal solve rate will be.

Arm	Method	Question it answers
A0	Base Gemma 4 31B	Starting point
A1	Base + prompt "be efficient, budget N calls"	Is training needed at all?
A2	RFT on shortest successful trajectories	Does simple distillation match RL?
A3	GRPO with R1	Does outcome-only RL cut cost by accident?
A4	GRPO with R2, lambda sweep	Does cost-aware reward shift the frontier?
A5	GRPO with R3	Do verifiable process rewards help further?
A6	Step-DPO from R3-style pairs	Does DPO beat online RL, as in PRLCoder?
A7	Best of A4 to A6 + Laya router	Does a zero-token System 1 add savings?
A8	Best arm + Gemma-LoRA router	Is the router submittable under the rules?
Ablations	Ungated reward; no NLL anchor; efficiency-only pairs; no potential shaping; lambda in 6 steps; 3 seeds	What actually matters

Metrics.

- Resolve rate on the 129 dev tasks, 5 samples each.
- Cost per solved task: tool calls, generated plus observation tokens, unique files, tests run, duplicate reads.
- Time to first correct localization: the first step t where the seen set intersects S*.
- Localization precision: share of reads with positive graph relevance.
- Wall-clock time and GPU-seconds per solved task.

Pareto analysis. Plot resolve rate against mean interactions for every arm and lambda, and report hypervolume relative to the base model. The target is movement toward the upper left.

Statistics. McNemar's test on paired solve outcomes, paired Wilcoxon and bootstrap confidence intervals for costs, all with seeds reported. For generalization, train with each repository held out in turn and evaluate on it (leave-one-repo-out across FastAPI, Requests, Rich and HTTPX).

What each metric and test is for

Item	What it measures	Why it is there
Resolve rate	Share of tasks whose hidden tests pass, over 5 samples each	The success side of the trade-off
Cost per solved task	Total cost over all attempts divided by the number of solves	Failed attempts are not hidden. It is the effective price of one solve
Tool calls, tokens, unique files, tests, duplicate reads	Raw counts per solved task	Readable numbers that do not depend on the weights in C
Time to first localization	First step at which the agent has seen any node in S*	Separates finding the right place from writing the right fix
Localization precision	Share of reads whose graph relevance is above zero	How focused the exploration is
Wall-clock and GPU-seconds	Real time and compute per solved task	Checks that the cost proxy matches real cost
Hypervolume	Area dominated by an arm's resolve-versus-interactions points, from a fixed reference point	One number for how far the frontier moved up and to the left
McNemar's test	Paired test on tasks one arm solved and the other did not	Both arms see the same tasks, so only the disagreements carry information
Wilcoxon and bootstrap intervals	Paired differences in cost per task, and a range from resampling tasks	Costs are skewed, so a rank test and resampling are safer than a t-test

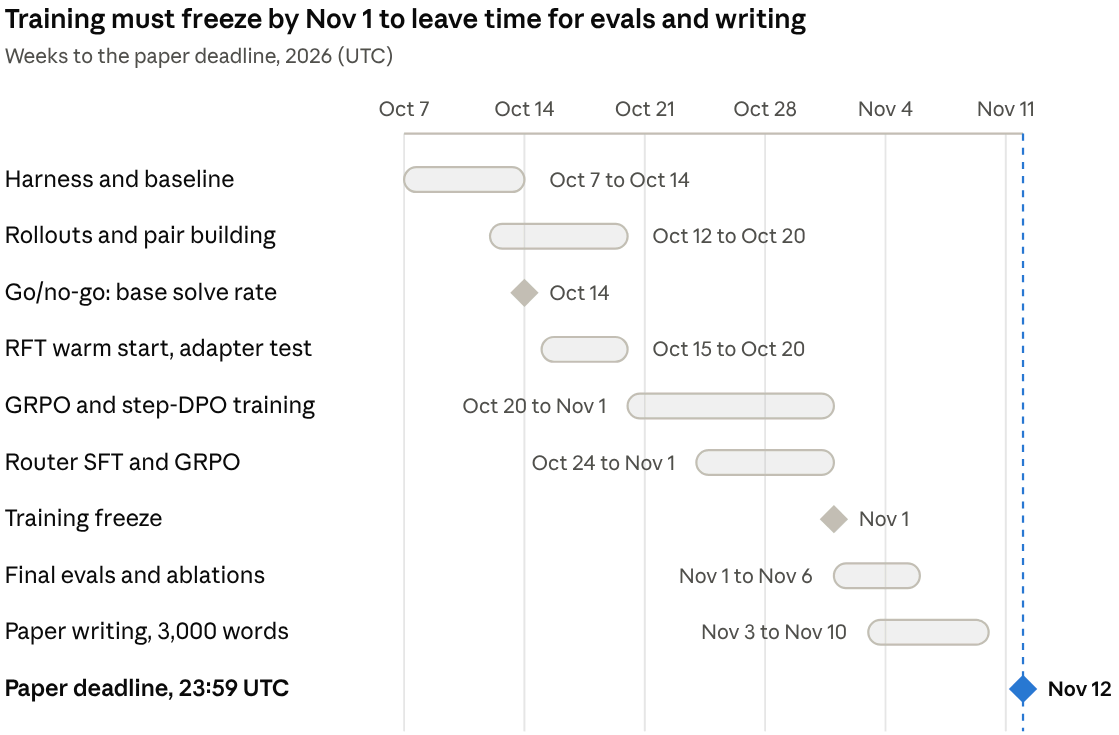
Item	What it measures	Why it is there
Non-inferiority margin	Largest solve-rate drop you accept, fixed before running	Turns "about the same solve rate" into a testable claim
Leave-one-repo-out	Train without one repository, then test on it	The hidden set comes from private repositories, so this checks transfer to unseen code

A warning on the margin: with 129 tasks, the standard error of a solve rate near 50% is about 4.4 points, so a margin much smaller than 5 points cannot be resolved. The paired comparison is tighter but of the same order.

Why each ablation exists.

Ablation	Question it answers
Ungated reward	Does early-quit collapse really happen?
No NLL anchor	Does DPO need the anchor?
Efficiency-only pairs	Do competence pairs protect the solve rate?
No potential shaping	Is R3 better than R2, or is the bonus noise?
Lambda in six steps	Where is the frontier, and where does solve rate fall off?
Three seeds	How much of any difference is run-to-run noise?

Schedule



schedule · 5 weeks, 2 gates

Training freezes on Nov 1 so evals and writing fit before the Nov 12 deadline; aim to submit by Nov 10 for a buffer. At the Oct 14 gate, if the base solve rate on the dev set is too low to yield successful trajectories, switch to the curriculum fallback in the risks table.

Two-week execution plan

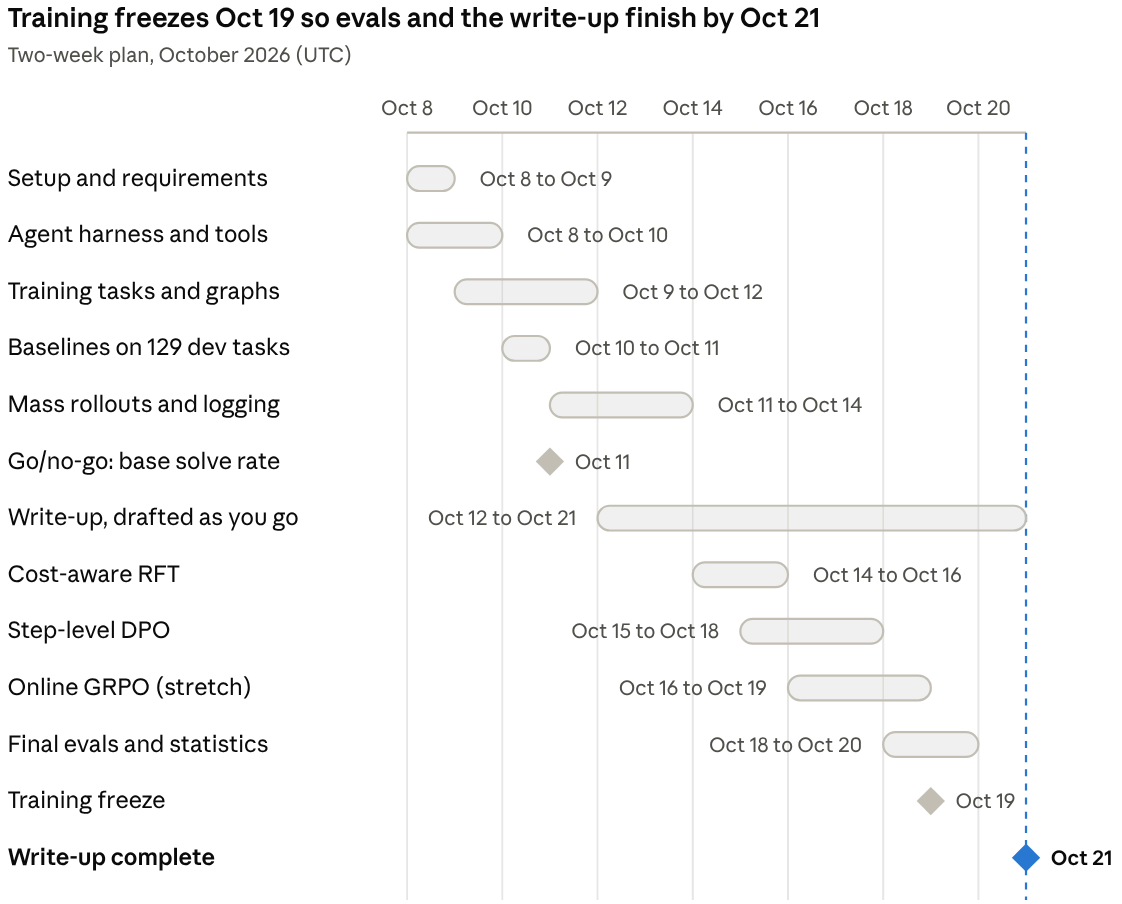
This plan targets a finished write-up on Oct 21 and keeps Nov 12 as buffer. It replaces the pace of the five-week schedule above, not the real deadline.

Honest odds, once. In my judgment the full plan, including online GRPO on the 31B model, has roughly a 20% chance of finishing in 14 days. A complete paper built on baselines, cost-aware fine-tuning, step-level DPO and the Pareto analysis has roughly 60%. The time goes to infrastructure, not training: stable multi-turn tool calling on Gemma 4 31B, thousands of per-task Docker environments, and code graphs for training tasks. Four things I could not verify:

- The Kaggle overview and rules pages did not load for me, so the harness details, external-data rules and release requirements need your own read on day 1.

- Whether verl or SkyRL run Gemma 4 with LoRA. Unsloth's Gemma 4 guide documents GRPO only with vLLM fast inference switched off, which is slower.
- A Hugging Face report shows Gemma 4 12B looping on long agent prompts with many tools. It is not reported for the 31B, but test it on day 2.
- Whether the extra three weeks to Nov 12 can absorb GRPO if it slips.

So the plan is built in tiers. Each tier is a complete paper, and each later tier upgrades it. Nothing is wasted if you stop early.



two-week plan · 13 workstreams, 2 gates

Write-up runs in parallel from Oct 12. The Oct 11 gate decides whether to train on easier tasks first. The Oct 19 freeze is firm: whatever has finished by then is what gets reported.

Tiers.

Tier	What ships	Arms	Claim it supports
1 (must)	Harness, baselines, cost-aware RFT, Pareto, write-up	A0, A1, A2 with R1, R2 and R3 selection	Choosing training trajectories by cost-aware reward shifts the success-versus-cost frontier
2	Step-level DPO	A6	Step pairs improve efficiency beyond fine-tuning on whole trajectories
3	Online GRPO	A3 to A5	Online RL with a cost-aware reward beats outcome-only RL
4	Laya router	A7, A8	A zero-token router adds further savings

A2 here means supervised fine-tuning on filtered trajectories, selected by each reward. It is offline and needs no RL infrastructure, yet it still compares reward formulations. That is why it is Tier 1.

Requirements checklist.

Requirement	Detail	Done when
Kaggle account, team, rules	Read both tracks' overview and rules pages; teams up to five	Rules summary committed to the repo
GPUs	One group serving the 31B quantized model through vLLM (tensor parallel, I would assume 4 x 80 GB) and a separate group for training; long multi-turn contexts need 80 GB class cards	Both groups reachable, and a 16k-token context fits
Storage	Per-repo Docker images for training tasks plus rollout logs; plan for hundreds of GB	Disk quota confirmed

Requirement	Detail	Done when
Docker	Needed by SWE-smith and SWE-rebench environments; SWE-smith was tested on Ubuntu 22.04 and does not plan Windows or macOS support	Containers start on the cluster
Offline execution	The competition runs without internet, so dependencies must come from the provided wheels	Dev tasks pass their reference patch offline
Licenses	SWE-smith toolkit is MIT; check each dataset's license and the competition's external-data conditions	One line per dataset in the write-up
Experiment tracking	Weights and Biases or MLflow, fixed seeds, one config file per arm	Any table regenerates from logged runs

Steps in sequence

Step 0. Setup and requirements (Oct 7 to 8).

- Do: read the overview and rules pages for both Kaggle tracks in a browser and note the word limit, external-data rules, release requirements and submission counts. Download the competition data with the Kaggle CLI: `tasks.jsonl`, `snapshots/`, `graphs/`, `embeddings/`, `wheels/`. Create the repo, the experiment-tracking project and one config file per arm.
- Libraries: Kaggle CLI, git, uv, Weights and Biases, YAML configs.
- Done when: the 129 dev tasks load, the rules summary is committed, and both GPU groups are reserved.

Step 1. Serve the model and build the harness (Oct 8 to 10).

- Do: serve `gemma-4-31b-it-qat-w4a16-ct` with vLLM using tensor parallelism, prefix caching and the OpenAI-compatible server, with the Gemma 4 chat and tool-call settings from the vLLM docs. Build the agent loop to mimic the competition harness, which is defined through the Agent Development Kit agent config: same tool names, argument formats and observation formats, because an adapter trained here must run there. Tools: search (ripgrep), read_span (path, start, end), graph_neighbors

(NetworkX over `graphs/`), semantic_search (cosine similarity over `embeddings/`), edit (exact-string replace), run_tests (pytest using the provided wheels, no network), submit. Wrap every call in a logger that writes one step record to JSONL, with prompt, generated and observation tokens counted separately.

- Libraries: vLLM, the OpenAI Python client, Agent Development Kit, ripgrep, NetworkX, NumPy, pytest, pydantic.
- Done when: five dev tasks run end to end with complete logs and no looping on long tool prompts. If loops appear, shorten the system prompt and tool schemas first.

Step 2. Baselines and calibration (Oct 10 to 11).

- Do: run A0 (base model) and A1 (base model with a prompt-level call budget) on all 129 dev tasks, 5 samples each. From the successful runs compute the per-task medians n-bar. Fit the weights w_k by non-negative regression of measured GPU-seconds on the counts, set c_max near the 90th to 95th percentile of the baseline ratios, and choose the lambda grid.
- Libraries: pandas, scikit-learn (`LinearRegression(positive=True)`), SciPy, matplotlib.
- Done when: a table of solve rate and cost per solved task exists for A0 and A1. Gate on Oct 11: if the base solve rate on dev is below about 10% (my threshold, not a rule), start training on easier tasks and move to harder ones in stages.

Step 3. Training tasks and code graphs (Oct 9 to 12, in parallel with steps 1 and 2).

- Do: take Python tasks from SWE-smith (MIT-licensed toolkit, about 52k synthetic task instances, one Docker image per repository) and from SWE-rebench V2 (32,000+ tasks across 20 languages, with pre-built images). Exclude FastAPI, Requests, Rich and HTTPX so the dev set stays held out. Keep 1,000 to 2,000 tasks whose containers start, whose tests fail before the reference patch and pass after it. Build a code graph per repository in the same schema as the provided `graphs/` (read the dataset description first; parse with `ast` or tree-sitter and store with NetworkX). Extract the symbol set S* from each reference patch: unidiff for the hunks, `ast` for the enclosing functions and classes. Precompute the relevance map for the progress score.
- Libraries: Hugging Face `datasets`, the Docker SDK for Python, `swesmith`, the SWE-bench harness, tree-sitter, NetworkX, unidiff.
- Done when: at least 1,000 validated tasks have a graph and S* on disk. The graph schema on training tasks must match the one the agent sees on dev tasks, or the graph tool will behave differently at test time.

Step 4. Mass rollouts (Oct 11 to 14).

- Do: run 16 attempts per training task with the base model through vLLM against a pool of containers. Log every step. Offline, compute C, the progress score and R1, R2 and R3 for each trajectory. My target is at least 10,000 trajectories with at least 1,000 solved.
- Libraries: Ray or asyncio with docker-py, SWE-ReX for remote execution, vLLM, PyArrow and Parquet for the trajectory table.
- Done when: the trajectory table is queryable and solve rate by task difficulty is plotted.

Step 5. Cost-aware RFT, Tier 1 (Oct 14 to 16).

- Do: build three training sets. R1: all successful trajectories. R2: the lowest-cost successes per task, for example the best 2 of 16 by R2. R3: the best by R3. Train one LoRA on each, with loss only on tokens Gemma wrote. Load each adapter into vLLM on the quantized model and confirm it still helps on day one of this step.
- Libraries: TRL `SFTTrainer`, PEFT, Unsloth (its guide lists Gemma 4 31B QLoRA at about 22 GB, and long contexts need more), Accelerate or DeepSpeed.
- Done when: three adapters exist and each has a dev evaluation.

Step 6. Step-level DPO, Tier 2 (Oct 15 to 18).

- Do: choose branch states in solved trajectories, preferably just before the first correct localization. Rebuild the state by replaying the earlier tool calls in a fresh container, or by snapshotting with `docker commit`. Sample K = 8 next actions, run m = 8 continuations from each, compute the step value with R3, and keep pairs that pass the margin and at-least-one-solve rules. Train DPO on the pairs.
- Libraries: TRL `DPOTrainer` (prompt, chosen and rejected fit a shared-prefix pair directly, and its `rpo_alpha` option adds an NLL term; check the current docs for the exact name), PEFT.
- Done when: at least 2,000 pairs exist and a DPO adapter has a dev evaluation. Budget 64 rollouts per branch state.

Step 7. Online GRPO, Tier 3, stretch (smoke test by Oct 14, training Oct 16 to 19).

- Do: first run a 5-step LoRA GRPO smoke test with Gemma 4 on SkyRL or verl-agent. If it does not run by Oct 14, drop GRPO from the two-week target and revisit it before Nov 9. Otherwise write the environment so observations are tool outputs and the reward comes from the logger, start from the R2 fine-tuned adapter, and use G = 16.
- Libraries: SkyRL (built on verl and OpenHands, v0.3.0 released July 2026, environments are text-in text-out classes), verl-agent (supports LoRA and GRPO, and adds GiGPO, which gives step-level credit and speaks directly to the caveat on dense credit), Unsloth GRPO (for Gemma 4 it needs vLLM fast inference off, so it is slower).

- Done when: reward rises, KL stays stable, and a checkpoint has a dev evaluation. If only a smaller Gemma 4 model trains, report it as a scale-down and say so plainly.

Step 8. Final evaluation (Oct 18 to 20).

- Do: freeze training on Oct 19. Run every arm on all 129 dev tasks, 5 samples each, in the exact harness. Compute the metrics, the Pareto plot, McNemar's test, Wilcoxon tests and bootstrap intervals. Run the cheap ablations: ungated reward in the RFT selection, and no NLL anchor in DPO. Because training uses only external tasks, the dev repositories are already held out, so leave-one-repo-out is optional now.
- Libraries: statsmodels (`mcnemar`), SciPy (`wilcoxon`), NumPy bootstrap, matplotlib or seaborn.
- Done when: every table and figure regenerates from one script.

Step 9. Write-up (drafted from Oct 12, complete Oct 21). Details in the next section.

Write-up plan

The paper track accepts at most 3,000 words in a Kaggle Writeup covering abstract, introduction, methods, experiments, related work and citations. Judges score five equally weighted criteria from 0 to 5. These details come from a secondary summary, so confirm them on the rules page.

Section	Words	Content	Write on
Abstract	150	Problem, method, headline numbers	Oct 20, after results
Introduction	350	Outcome-only rewards hide the cost gap; the question; contributions	Oct 12
Related work	300	PRLCoder (process supervision, DPO versus PPO), SWE-smith, SWE-rebench V2, SkyRL, verl-agent and GiGPO, laya-conductor	Oct 12 to 13
Method	800	Decision process, cost, R1 to R3, progress score, step pairs, training stages	Oct 13 to 15, adapted from this document

Section	Words	Content	Write on
Experiments	900	Setup, arms, Pareto figure, results table with intervals, ablations, statistics	Oct 19 to 21
Limitations	200	Small dev set, assumed weights, gold patch used in training, synthetic training tasks, one model	Oct 20
Conclusion	100	What was shown and what was not	Oct 21
Total	2,800	Leaves about 200 words of margin for captions	

Figures and tables. Figure 1 is the pipeline diagram already in this document. Figure 2 is the Pareto plot of resolve rate against interactions per solved task, one point per arm and lambda, which is the main result. Table 1 lists the arms. Table 2 gives results with confidence intervals. Table 3 gives the ablations. Script all of them from one results file so a late rerun updates everything.

Mapping to the judging criteria.

Criterion	What to show
Novelty	Graph-verified process rewards with no learned reward model, and the success-gated cost reward with its failure analysis
Quality	Training on external tasks and testing on held-out repositories, with paired statistics
Relevance	Direct link to agentic software engineering and learning efficient search
Verifiability	Release the code, the per-arm configs and the step-labeled trajectory dataset; state the licenses
Clarity	One figure that carries the result, and the cost formula explained once in plain words

Submission mechanics. Submit a rough draft by Oct 14 to learn the Kaggle Writeup format; the summary says five submissions are allowed and unsubmitted drafts are ineligible. A public PDF can replace full text in the Writeup.

What to cut if behind, in this order. Router first, then GRPO, then DPO, then the extra ablations. Never cut the baselines, the calibration, the statistics or the write-up. A paper with Tier 1 only still has a clear claim: cost-aware selection of training trajectories moves the frontier.

Resources

Need	Resource
Method precedent	Process-Supervised RL for Code Generation
Training tasks	SWE-smith, SWE-rebench V2
Existing SWE trajectories (other models, for reference or warm-start ideas)	CoderForge-Preview, which lists trajectories with rewards from SWE-rebench, SWE-smith and R2E-Gym
Agentic RL framework	SkyRL docs, SkyRL announcement, verl-agent
Gemma 4 fine-tuning	Unsloth Gemma 4 guide (the version I retrieved is German)
Competition harness	Agent Development Kit agent config
Router recipe	laya-conductor
Known Gemma 4 agent issue	Looping report on the 12B model
Competition summary	AI Wiki summary, to be checked against the Kaggle pages

Code skeleton

Three pieces to write first: the step logger, the reward module, and the masked DPO loss. Everything else is orchestration.

```
# 1. Step schema logged by every tool wrapper
@dataclass
class Step:
    task_id: str; t: int
    prompt_tokens: int; gen_tokens: int; obs_tokens: int
    tool: str; args: dict
    files_touched: list[str]; symbols_touched: list[str]
    is_dup_read: bool
    phase: str            # router label
    gen_text: str         # Gemma's own tokens (the only tokens trained on)

# 2. Graph relevance from the provided graphs/ directory
def relevance(graph, gold_syms, gamma_g=0.5):
    dist = multi_source_bfs(graph, gold_syms)      # node -> hops to gold
patch
    return {v: gamma_g ** d for v, d in dist.items()}

# 3. Cost and reward (R1, R2, R3)
def cost(steps, nbar, w, cmax=3.0):
    n = dict(calls=len(steps),
             tokens=sum(s.obs_tokens + s.gen_tokens for s in steps),
             files=len({f for s in steps for f in s.files_touched}),
             tests=sum(s.tool == 'test' for s in steps),
             dups=sum(s.is_dup_read for s in steps))
    return sum(w[k] * min(n[k] / max(nbar[k], 1), cmax) / cmax for k in w)

def reward(S, steps, nbar, w, rel, lam=0.5, eta=0.1, gamma=1.0,
variant='R3'):
    if variant == 'R1':
        return float(S)
    R = S * (1 - lam * cost(steps, nbar, w))
    if variant == 'R3':
        phi, seen = [0.0], set()
        for s in steps:
            seen |= set(s.symbols_touched)
            phi.append(max([rel.get(v, 0.0) for v in seen] or [0.0]))
        R += eta * sum(gamma * phi[i + 1] - phi[i] for i in
range(len(steps)))
    return R

# 4. DPO term: log-prob over Gemma-generated tokens only
def masked_logp(model, input_ids, action_mask):
```

```
    logits = model(input_ids).logits[:, :-1]
    lp = torch.log_softmax(logits, -1).gather(-1, input_ids[:, 1:,
None]).squeeze(-1)
    return (lp * action_mask[:, 1:]).sum(-1)
```

Pair builder, in words. For each branch state, sample K actions, run m rollouts per action, compute the mean reward, and emit (a+, a-) when the gap exceeds delta. Store the shared prefix once and tokenize chosen and rejected from the same prefix so the masks line up.

What each piece of the code does.

Piece	What it does	Why it is written this way
Step	One logged tool call with its token counts, files, symbols and the model's own text	The cost, the progress score and the training targets all come from this record
Prompt, generated and observation tokens kept separately	Counts for the context, the model's output and the tool output	Reading a huge file costs context even though the model wrote little
is_dup_read	True when a file span already read is read again with no edit between	Feeds the duplicate-read cost
gen_text	The tokens Gemma wrote	The only tokens DPO and GRPO train on
relevance	Breadth-first search outward from the reference patch's symbols, returning gamma_g to the power of the hop distance for each node	Builds Phi's lookup table once per task
cost	The formula for C, with the baseline floored at 1	The floor avoids dividing by zero when a baseline count is 0, such as duplicate reads
reward	R1 returns success only, R2 discounts by cost, R3 adds the summed progress bonus	One function for all three variants, so only a flag changes between arms

Piece	What it does	Why it is written this way
Default values	lam 0.5, eta 0.1, gamma 1.0	They satisfy eta below 1 minus lambda. Starting points to tune
masked_logp	Sum of log-probabilities over Gemma-written tokens only	DPO needs the sequence log-probability of each action under the policy and the reference

Risks and fallbacks

Risk	Why it matters	Fallback
Base solve rate too low	The hidden set is filtered for difficulty, so few successful trajectories exist	Train on easier external tasks first, curriculum by task difficulty
Early-quit collapse	Ungated cost rewards pay short failures	Success-gated R2 and R3; report the ungated ablation as a finding
Reward hacking via cheap tools	Agent may read tiny spans to cut tokens, then guess	Gate on S; add a minimum-context rule in the editor tool
Adapter does not transfer to the quantized checkpoint	Training precision differs from the competition model	Day-1 transfer test; train directly against the quantized weights if needed
Router disallowed in submission	Rules require Gemma for every agent and subagent	Distill into a Gemma LoRA subagent; keep Laya as a paper-only result
External data licensing	The rules restrict external data by accessibility and licence	Check each dataset licence; document sources in the paper
Train/test leakage	Dev tasks come from FastAPI, Requests, Rich, HTTPX	Exclude those repositories from training; evaluate leave-one-repo-out

Risk	Why it matters	Fallback
Wall-clock, not GPU	Sandbox startup and test execution cap rollout throughput	Parallelize containers, cache environments, reuse prefix KV

Sources

- Process-Supervised Reinforcement Learning for Code Generation (Ye et al., EMNLP 2025)
- Gemma 4 Developer Agent Competition summary (AI Wiki, 3 Oct 2026)
- laya-conductor model card (Hugging Face)
- Classify prompts with Laya, the open Jev (DEV Community)

The Kaggle overview pages did not render in my fetch, so competition rules (deadlines, model constraint, 3,000-word limit, tool restrictions) come from the AI Wiki summary. Confirm them on the paper-track rules page before you commit.